claude-windows / a9fd4426-f234-4e4a-bfb4-1f2c8255c661

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-muneem\a9fd4426-f234-4e4a-bfb4-1f2c8255c661\subagents\workflows\wf_5070b515-30d\agent-aab279f875dbd208a.jsonl`
- SHA-256: `cd61f27f34dba37473092aac54b27b8490a4d1e7239889d09bb316a75c674a40`
- Source modified: `2026-05-31T14:31:49+00:00`
- Imported at: `2026-07-05T16:48:21+00:00`
- project: `wf_5070b515-30d`
- session_id: `a9fd4426-f234-4e4a-bfb4-1f2c8255c661`

Transcript

1. user (2026-05-31T14:27:51.941Z)

Project context - muneem: a SINGLE-USER, LOCAL-FIRST personal-finance bookkeeper in Python (3.11-3.13),
developed on Windows, must also run on Linux. It parses bank/credit-card/investment statement PDFs into a
LOCAL SQLite ledger (stdlib sqlite3 today; schema v4: documents/offers/per-bank txn tables). The user now
wants to store REAL financial numbers and run BASIC SQL QUERIES (SELECT/WHERE/aggregations) over them, and
wants the database "hardened" (encryption-at-rest).

HARD CONSTRAINTS (a tool that violates these is unusable):
1. 100% LOCAL. No cloud KMS/SaaS. Key stays on the user machine (OS keychain via the "keyring" lib is available).
2. Usable from Python, ideally close to the stdlib sqlite3 API. Must KEEP SQL QUERYABILITY (WHERE/ORDER BY/
   aggregates over stored numbers) - so TRANSPARENT full-DB page encryption is strongly preferred over per-field
   app-level encryption that breaks queries.
3. License MUST be permissive & commercially safe: MIT / BSD / Apache-2.0 / ISC / HPND - for BOTH the Python
   binding AND every native dependency. AVOID/FLAG anything GPL/LGPL/AGPL. (PyMuPDF AGPL is already banned here.)
   OpenSSL is acceptable.
4. Cross-platform prebuilt WHEELS (Windows + Linux, cp311-cp313) strongly preferred - avoid a from-source C/JVM
   build burden.

Cite authoritative sources (project LICENSE files, PyPI pages, SPDX, official docs) with URLs.

ANGLE: Threat model, key management, and OS full-disk-encryption comparison.
Build the THREAT MODEL for at-rest encryption of a local personal-finance SQLite DB on a SINGLE-USER machine (dev Windows, also Linux). Address head-on: (a) if the KEY lives on the SAME machine (OS keychain / DPAPI / Keychain / Secret Service via Python keyring), what attacks does DB encryption actually STOP vs NOT (stolen/ discarded disk, backups synced to cloud, malware running as the user, shoulder-surf)? (b) How does this compare to relying on OS FULL-DISK ENCRYPTION (BitLocker / LUKS / FileVault)? Is FDE sufficient, complementary, or insufficient alone? (c) Best practice for KEY DERIVATION & STORAGE: passphrase+KDF (PBKDF2/Argon2/scrypt) vs random key in keychain; key rotation; what to avoid (key in source/config). (d) Net recommendation: does DB-level encryption add real marginal security over FDE+keychain for THIS threat model, or is it mostly defense-in-depth? Cite URLs/standards (e.g. OWASP, NIST).

2. assistant (2026-05-31T14:27:55.297Z)

I'll help you build a comprehensive threat model and analysis for at-rest encryption of a local

personal-finance SQLite database. Let me search the muneem project structure first to understand the current state, then conduct deep research on encryption approaches.

3. user (2026-05-31T14:27:56.249Z)

```
.venv\Lib\site-packages\pip\__init__.py
.venv\Lib\site-packages\pip\__main__.py
.venv\Lib\site-packages\pip\__pip-runner__.py
.venv\Lib\site-packages\pip\_internal\__init__.py
.venv\Lib\site-packages\pip\_internal\build_env.py
.venv\Lib\site-packages\pip\_internal\cache.py
.venv\Lib\site-packages\pip\_internal\configuration.py
.venv\Lib\site-packages\pip\_internal\exceptions.py
.venv\Lib\site-packages\pip\_internal\main.py
.venv\Lib\site-packages\pip\_internal\pyproject.py
.venv\Lib\site-packages\pip\_internal\self_outdated_check.py
.venv\Lib\site-packages\pip\_internal\cli\__init__.py
.venv\Lib\site-packages\pip\_internal\wheel_builder.py
.venv\Lib\site-packages\pip\_internal\cli\autocompletion.py
.venv\Lib\site-packages\pip\_internal\cli\base_command.py
.venv\Lib\site-packages\pip\_internal\cli\cmdoptions.py
.venv\Lib\site-packages\pip\_internal\cli\command_context.py
.venv\Lib\site-packages\pip\_internal\cli\index_command.py
.venv\Lib\site-packages\pip\_internal\cli\main.py
.venv\Lib\site-packages\pip\_internal\cli\main_parser.py
.venv\Lib\site-packages\pip\_internal\cli\parser.py
.venv\Lib\site-packages\pip\_internal\cli\progress_bars.py
.venv\Lib\site-packages\pip\_internal\cli\req_command.py
.venv\Lib\site-packages\pip\_internal\cli\spinners.py
.venv\Lib\site-packages\pip\_internal\cli\status_codes.py
.venv\Lib\site-packages\pip\_internal\commands\cache.py
.venv\Lib\site-packages\pip\_internal\commands\__init__.py
.venv\Lib\site-packages\pip\_internal\commands\check.py
.venv\Lib\site-packages\pip\_internal\commands\completion.py
.venv\Lib\site-packages\pip\_internal\commands\configuration.py
.venv\Lib\site-packages\pip\_internal\commands\debug.py
.venv\Lib\site-packages\pip\_internal\commands\download.py
.venv\Lib\site-packages\pip\_internal\commands\freeze.py
.venv\Lib\site-packages\pip\_internal\commands\hash.py
.venv\Lib\site-packages\pip\_internal\commands\help.py
.venv\Lib\site-packages\pip\_internal\commands\index.py
.venv\Lib\site-packages\pip\_internal\commands\inspect.py
.venv\Lib\site-packages\pip\_internal\commands\install.py
.venv\Lib\site-packages\pip\_internal\commands\list.py
.venv\Lib\site-packages\pip\_internal\commands\lock.py
.venv\Lib\site-packages\pip\_internal\commands\search.py
.venv\Lib\site-packages\pip\_internal\commands\show.py
.venv\Lib\site-packages\pip\_internal\commands\uninstall.py
.venv\Lib\site-packages\pip\_internal\commands\wheel.py
.venv\Lib\site-packages\pip\_internal\distributions\__init__.py
.venv\Lib\site-packages\pip\_internal\distributions\base.py
.venv\Lib\site-packages\pip\_internal\distributions\installed.py
.venv\Lib\site-packages\pip\_internal\distributions\sdist.py
.venv\Lib\site-packages\pip\_internal\distributions\wheel.py
.venv\Lib\site-packages\pip\_internal\index\__init__.py
.venv\Lib\site-packages\pip\_internal\index\collector.py
.venv\Lib\site-packages\pip\_internal\index\package_finder.py
.venv\Lib\site-packages\pip\_internal\index\sources.py
.venv\Lib\site-packages\pip\_internal\locations\__init__.py
.venv\Lib\site-packages\pip\_internal\locations\_distutils.py
.venv\Lib\site-packages\pip\_internal\locations\_sysconfig.py
.venv\Lib\site-packages\pip\_internal\locations\base.py
.venv\Lib\site-packages\pip\_internal\metadata\_json.py
.venv\Lib\site-packages\pip\_internal\metadata\__init__.py
.venv\Lib\site-packages\pip\_internal\metadata\base.py
```

```
.venv\Lib\site-packages\pip\_internal\metadata\pkg_resources.py
.venv\Lib\site-packages\pip\_internal\metadata\importlib\__init__.py
.venv\Lib\site-packages\pip\_internal\metadata\importlib\_compat.py
.venv\Lib\site-packages\pip\_internal\metadata\importlib\_dists.py
.venv\Lib\site-packages\pip\_internal\metadata\importlib\_envs.py
.venv\Lib\site-packages\pip\_internal\models\__init__.py
.venv\Lib\site-packages\pip\_internal\models\candidate.py
.venv\Lib\site-packages\pip\_internal\models\direct_url.py
.venv\Lib\site-packages\pip\_internal\models\format_control.py
.venv\Lib\site-packages\pip\_internal\models\index.py
.venv\Lib\site-packages\pip\_internal\models\installation_report.py
.venv\Lib\site-packages\pip\_internal\models\link.py
.venv\Lib\site-packages\pip\_internal\models\release_control.py
.venv\Lib\site-packages\pip\_internal\models\scheme.py
.venv\Lib\site-packages\pip\_internal\models\search_scope.py
.venv\Lib\site-packages\pip\_internal\models\selection_prefs.py
.venv\Lib\site-packages\pip\_internal\models\target_python.py
.venv\Lib\site-packages\pip\_internal\models\wheel.py
.venv\Lib\site-packages\pip\_internal\network\auth.py
.venv\Lib\site-packages\pip\_internal\network\__init__.py
.venv\Lib\site-packages\pip\_internal\network\cache.py
.venv\Lib\site-packages\pip\_internal\network\download.py
.venv\Lib\site-packages\pip\_internal\network\lazy_wheel.py
.venv\Lib\site-packages\pip\_internal\network\session.py
.venv\Lib\site-packages\pip\_internal\network\utils.py
.venv\Lib\site-packages\pip\_internal\network\xmlrpc.py
.venv\Lib\site-packages\pip\_internal\operations\check.py
.venv\Lib\site-packages\pip\_internal\operations\__init__.py
.venv\Lib\site-packages\pip\_internal\operations\freeze.py
.venv\Lib\site-packages\pip\_internal\operations\prepare.py
.venv\Lib\site-packages\pip\_internal\operations\build\build_tracker.py
.venv\Lib\site-packages\pip\_internal\operations\build\__init__.py
.venv\Lib\site-packages\pip\_internal\operations\build\metadata.py
.venv\Lib\site-packages\pip\_internal\operations\build\metadata_editable.py
.venv\Lib\site-packages\pip\_internal\operations\build\wheel.py
.venv\Lib\site-packages\pip\_internal\operations\build\wheel_editable.py
.venv\Lib\site-packages\pip\_internal\operations\install\__init__.py
.venv\Lib\site-packages\pip\_internal\operations\install\wheel.py
.venv\Lib\site-packages\pip\_internal\req\constructors.py
.venv\Lib\site-packages\pip\_internal\req\__init__.py
(Results are truncated. Consider using a more specific path or pattern.)
```

4. user (2026-05-31T14:27:56.349Z)

```
.venv\Lib\site-packages\pip\_vendor\cachecontrol\LICENSE.txt
.venv\Lib\site-packages\pip\_vendor\certifi\LICENSE
.venv\Lib\site-packages\pip\_vendor\distlib\LICENSE.txt
.venv\Lib\site-packages\pip\_vendor\distro\LICENSE
.venv\Lib\site-packages\pip\_vendor\idna\LICENSE.md
.venv\Lib\site-packages\pip\_vendor\packaging\LICENSE
.venv\Lib\site-packages\pip\_vendor\packaging\LICENSE.APACHE
.venv\Lib\site-packages\pip\_vendor\packaging\LICENSE.BSD
.venv\Lib\site-packages\pip\_vendor\pkg_resources\LICENSE
.venv\Lib\site-packages\pip\_vendor\platformdirs\LICENSE
.venv\Lib\site-packages\pip\_vendor\pygments\LICENSE
.venv\Lib\site-packages\pip\_vendor\pyproject_hooks\LICENSE
.venv\Lib\site-packages\pip\_vendor\requests\LICENSE
.venv\Lib\site-packages\pip\_vendor\resolvelib\LICENSE
.venv\Lib\site-packages\pip\_vendor\rich\LICENSE
.venv\Lib\site-packages\pip\_vendor\tomli\LICENSE
.venv\Lib\site-packages\pip\_vendor\tomli_w\LICENSE
.venv\Lib\site-packages\pip\_vendor\truststore\LICENSE
.venv\Lib\site-packages\pip\_vendor\urllib3\LICENSE.txt
.venv\Lib\site-packages\pip-26.1.1.dist-info\licenses\LICENSE.txt
.venv\Lib\site-packages\pip-26.1.1.dist-info\licenses\src\pip\_vendor\cachecontrol\LICENSE.txt
```

.venv\Lib\site-packages\pip-26.1.1.dist-info\licenses\src\pip_vendor\certifi\LICENSE
.venv\Lib\site-packages\pip-26.1.1.dist-info\licenses\src\pip_vendor\distlib\LICENSE.txt
.venv\Lib\site-packages\pip-26.1.1.dist-info\licenses\src\pip_vendor\distro\LICENSE
.venv\Lib\site-packages\pip-26.1.1.dist-info\licenses\src\pip_vendor\idna\LICENSE.md
.venv\Lib\site-packages\pip-26.1.1.dist-info\licenses\src\pip_vendor\packaging\LICENSE
.venv\Lib\site-packages\pip-26.1.1.dist-info\licenses\src\pip_vendor\packaging\LICENSE.APACHE
.venv\Lib\site-packages\pip-26.1.1.dist-info\licenses\src\pip_vendor\packaging\LICENSE.BSD
.venv\Lib\site-packages\pip-26.1.1.dist-info\licenses\src\pip_vendor\pkg_resources\LICENSE
.venv\Lib\site-packages\pip-26.1.1.dist-info\licenses\src\pip_vendor\platformdirs\LICENSE
.venv\Lib\site-packages\pip-26.1.1.dist-info\licenses\src\pip_vendor\pygments\LICENSE
.venv\Lib\site-packages\pip-26.1.1.dist-info\licenses\src\pip_vendor\pyproject_hooks\LICENSE
.venv\Lib\site-packages\pip-26.1.1.dist-info\licenses\src\pip_vendor\requests\LICENSE
.venv\Lib\site-packages\pip-26.1.1.dist-info\licenses\src\pip_vendor\resolvelib\LICENSE
.venv\Lib\site-packages\pip-26.1.1.dist-info\licenses\src\pip_vendor\rich\LICENSE
.venv\Lib\site-packages\pip-26.1.1.dist-info\licenses\src\pip_vendor\tomli\LICENSE
.venv\Lib\site-packages\pip-26.1.1.dist-info\licenses\src\pip_vendor\tomli_w\LICENSE
.venv\Lib\site-packages\pip-26.1.1.dist-info\licenses\src\pip_vendor\truststore\LICENSE
.venv\Lib\site-packages\pip-26.1.1.dist-info\licenses\src\pip_vendor\urllib3\LICENSE.txt
.venv\Lib\site-packages\shellingham-1.5.4.dist-info\LICENSE
.venv\Lib\site-packages\ruff-0.15.15.dist-info\licenses\LICENSE
.venv\Lib\site-packages\pywin32_ctypes-0.2.3.dist-info\LICENSE.txt
.venv\Lib\site-packages\pygments-2.20.0.dist-info\licenses\LICENSE
.venv\Lib\site-packages\pluggy-1.6.0.dist-info\licenses\LICENSE
.venv\Lib\site-packages\platformdirs-4.10.0.dist-info\licenses\LICENSE
.venv\Lib\site-packages\packaging-26.2.dist-info\licenses\LICENSE
.venv\Lib\site-packages\packaging-26.2.dist-info\licenses\LICENSE.APACHE
.venv\Lib\site-packages\packaging-26.2.dist-info\licenses\LICENSE.BSD
.venv\Lib\site-packages\more_itertools-11.1.0.dist-info\licenses\LICENSE
.venv\Lib\site-packages\mdurl-0.1.2.dist-info\LICENSE
.venv\Lib\site-packages\jaraco_context-6.1.2.dist-info\licenses\LICENSE
.venv\Lib\site-packages\iniconfig-2.3.0.dist-info\licenses\LICENSE
.venv\Lib\site-packages\colorama-0.4.6.dist-info\licenses\LICENSE.txt
.venv\Lib\site-packages\annotated_doc-0.0.4.dist-info\licenses\LICENSE
.venv\Lib\site-packages\pytest-9.0.3.dist-info\licenses\LICENSE
.venv\Lib\site-packages\markdown_it_py-4.2.0.dist-info\licenses\LICENSE
.venv\Lib\site-packages\markdown_it_py-4.2.0.dist-info\licenses\LICENSE.markdown-it
.venv\Lib\site-packages\jaraco_functools-4.5.0.dist-info\licenses\LICENSE
.venv\Lib\site-packages\jaraco.classes-3.4.0.dist-info\LICENSE
.venv\Lib\site-packages\rich-15.0.0.dist-info\licenses\LICENSE
.venv\Lib\site-packages\keyring-25.7.0.dist-info\licenses\LICENSE
.venv\Lib\site-packages\typer-0.26.4.dist-info\licenses\LICENSE
.venv\Lib\site-packages\typer_click\LICENSE.txt
.venv\Lib\site-packages\pyyaml-6.0.3.dist-info\licenses\LICENSE
.venv\Lib\site-packages\pycparser-3.0.dist-info\licenses\LICENSE
.venv\Lib\site-packages\charset_normalizer-3.4.7.dist-info\licenses\LICENSE
.venv\Lib\site-packages\cffi-2.0.0.dist-info\licenses\LICENSE
.venv\Lib\site-packages\cryptography-48.0.0.dist-info\licenses\LICENSE
.venv\Lib\site-packages\cryptography-48.0.0.dist-info\licenses\LICENSE.APACHE
.venv\Lib\site-packages\cryptography-48.0.0.dist-info\licenses\LICENSE.BSD
.venv\Lib\site-packages\pillow-12.2.0.dist-info\licenses\LICENSE
.venv\Lib\site-packages\pytesseract-0.3.13.dist-info\LICENSE
.venv\Lib\site-packages\pdfminer_six-20251230.dist-info\licenses\LICENSE
.venv\Lib\site-packages\pdfplumber-0.11.9.dist-info\licenses\LICENSE.txt
.venv\Lib\site-packages\urllib3-2.7.0.dist-info\licenses\LICENSE.txt
.venv\Lib\site-packages\uritemplate-4.2.0.dist-info\licenses\LICENSE
.venv\Lib\site-packages\pypdf-6.12.2.dist-info\licenses\LICENSE
.venv\Lib\site-packages\pyparsing-3.3.2.dist-info\licenses\LICENSE
.venv\Lib\site-packages\pyasn1-0.6.3.dist-info\licenses\LICENSE.rst
.venv\Lib\site-packages\protobuf-7.35.0.dist-info\LICENSE
.venv\Lib\site-packages\oauthlib-3.3.1.dist-info\licenses\LICENSE
.venv\Lib\site-packages\idna-3.17.dist-info\licenses\LICENSE.md
.venv\Lib\site-packages\certifi-2026.5.20.dist-info\licenses\LICENSE
.venv\Lib\site-packages\requests-2.34.2.dist-info\licenses\LICENSE
.venv\Lib\site-packages\pyasn1_modules-0.4.2.dist-info\licenses\LICENSE.txt
.venv\Lib\site-packages\proto_plus-1.28.0.dist-info\licenses\LICENSE

```
.venv\Lib\site-packages\httplib2-0.31.2.dist-info\licenses\LICENSE
.venv\Lib\site-packages\googleapis_common_protos-1.75.0.dist-info\licenses\LICENSE
.venv\Lib\site-packages\requests_oauthlib-2.0.0.dist-info\LICENSE
.venv\Lib\site-packages\google_auth-2.53.0.dist-info\licenses\LICENSE
.venv\Lib\site-packages\google_auth_oauthlib-1.4.0.dist-info\licenses\LICENSE
.venv\Lib\site-packages\google_auth_httplib2-0.4.0.dist-info\licenses\LICENSE
.venv\Lib\site-packages\google_api_core-2.30.3.dist-info\licenses\LICENSE
.venv\Lib\site-packages\google_api_python_client-2.197.0.dist-info\licenses\LICENSE
```

5. user (2026-05-31T14:27:56.414Z)

```
No files found
```

6. user (2026-05-31T14:27:56.441Z)

```
pyproject.toml
```

7. assistant (2026-05-31T14:27:58.322Z)

```
Let me look at the project structure more directly:
```

8. user (2026-05-31T14:27:58.597Z)

```
1       [build-system]
2       requires = ["setuptools>=61"]
3       build-backend = "setuptools.build_meta"
4
5       [project]
6       name = "muneem"
7       description = "A local-first personal-finance bookkeeper that ingests financial
documents from email into a consolidated ledger."
8       readme = "README.md"
9       requires-python = ">=3.11"
10      dynamic = ["version"]
11      # Private, single-user tool ? this classifier makes packaging tools refuse to upload it
to PyPI.
12      classifiers = ["Private :: Do Not Upload"]
13      dependencies = [
14          "typer>=0.12",
15          "platformdirs>=4.0",
16          "keyring>=25.0",
17          "pdfminer.six>=20240706",
18          "pypdfium2>=4.30",
19          "pytesseract>=0.3.10",
20          "pillow>=10.0",
21          "pypdf>=4.0",
22          "google-api-python-client>=2.100",
23          "google-auth>=2.30",
24          "google-auth-oauthlib>=1.2",
25          "pdfplumber>=0.11",
26      ]
27
28      [project.scripts]
29      muneem = "muneem.cli:app"
30
31      [project.optional-dependencies]
32      outlook = ["msal>=1.0", "requests>=2.0"]
33      dev = [
34          "pytest>=8.0",
35          "ruff>=0.6",
36          "pyyaml>=6.0",
37      ]
38
39      [tool.setuptools.dynamic]
```

```
40      version = { attr = "muneem.__version__" }
41
42      [tool.setuptools.packages.find]
43      where = ["src"]
44
45      [tool.ruff]
46      line-length = 100
47      src = ["src", "tests"]
48
49      [tool.ruff.lint]
50      select = ["E", "F", "I", "UP", "B"]
51
52      [tool.ruff.format]
53      quote-style = "double"
54      indent-style = "space"
55      skip-magic-trailing-comma = false
56      line-ending = "auto"
57
58      [tool.ruff.lint.flake8-bugbear]
59      # Typer uses Option()/Argument() as parameter defaults by design.
60      extend-immutable-calls = ["typer.Option", "typer.Argument"]
61
62      [tool.pytest.ini_options]
63      # Default runs are offline and deterministic: the ocr/llm/local_statements tiers
64      # (see docs/TESTING.md) are opt-in and excluded here.
65      addopts = "-ra -m 'not ocr and not llm and not local_statements'"
66      testpaths = ["tests"]
67      markers = [
68          "ocr: requires the Tesseract binary (image-OCR tier)",
69          "llm: makes live LLM calls (never in default CI)",
70          "local_statements: reads gitignored real statements (local only)",
71      ]
72
```

9. user (2026-05-31T14:27:59.385Z)

```
C:\Users\avidu\Projects\muneem/.githooks/precommit_check.py
C:\Users\avidu\Projects\muneem/src/muneem/cli.py
C:\Users\avidu\Projects\muneem/src/muneem/config.py
C:\Users\avidu\Projects\muneem/src/muneem/db.py
C:\Users\avidu\Projects\muneem/src/muneem/extract.py
C:\Users\avidu\Projects\muneem/src/muneem/ingestion/base.py
C:\Users\avidu\Projects\muneem/src/muneem/ingestion/gmail.py
C:\Users\avidu\Projects\muneem/src/muneem/ingestion/imap.py
C:\Users\avidu\Projects\muneem/src/muneem/ingestion/outlook.py
C:\Users\avidu\Projects\muneem/src/muneem/ingestion/__init__.py
C:\Users\avidu\Projects\muneem/src/muneem/ocr.py
C:\Users\avidu\Projects\muneem/src/muneem/offers.py
C:\Users\avidu\Projects\muneem/src/muneem/parsers/au.py
C:\Users\avidu\Projects\muneem/src/muneem/parsers/axis.py
C:\Users\avidu\Projects\muneem/src/muneem/parsers/base.py
C:\Users\avidu\Projects\muneem/src/muneem/parsers/clustering.py
C:\Users\avidu\Projects\muneem/src/muneem/parsers/concepts.py
C:\Users\avidu\Projects\muneem/src/muneem/parsers/engine.py
C:\Users\avidu\Projects\muneem/src/muneem/parsers/hdfc.py
C:\Users\avidu\Projects\muneem/src/muneem/parsers/profiles.py
C:\Users\avidu\Projects\muneem/src/muneem/parsers/savings.py
C:\Users\avidu\Projects\muneem/src/muneem/parsers/validation.py
C:\Users\avidu\Projects\muneem/src/muneem/parsers/__init__.py
C:\Users\avidu\Projects\muneem/src/muneem/password_rules.py
C:\Users\avidu\Projects\muneem/src/muneem/secrets.py
C:\Users\avidu\Projects\muneem/src/muneem/statement_unlock.py
C:\Users\avidu\Projects\muneem/src/muneem/unlock.py
C:\Users\avidu\Projects\muneem/src/muneem/__init__.py
C:\Users\avidu\Projects\muneem/tests/conftest.py
```

C:\Users\avidu\Projects\muneem/tests/ingestion/test_gmail.py

10. assistant (2026-05-31T14:28:01.659Z)

Now let me examine the current database and secrets handling:

11. user (2026-05-31T14:28:01.886Z)

```
1       """Local SQLite store: the ``documents`` and ``offers`` slice of the schema.
2
3       This is the minimal slice the promo spine needs (ROADMAP A.2). The full schema
4       (accounts, transactions, holdings, merchants, items) grows with later milestones.
5       ``documents.sha256`` is UNIQUE so re-ingesting the same file is a no-op.
6       ``offers.source`` records provenance ('text' = exact text layer, 'ocr' = approximate).
7       """
8
9       from __future__ import annotations
10
11      import sqlite3
12      from collections.abc import Iterator
13      from contextlib import contextmanager
14      from pathlib import Path
15
16      SCHEMA_VERSION = 4
17
18      _SCHEMA = """
19      CREATE TABLE IF NOT EXISTS documents (
20          id            INTEGER PRIMARY KEY,
21          source        TEXT NOT NULL,
22          external_id   TEXT,
23          sender        TEXT,
24          subject       TEXT,
25          received_date TEXT,
26          doc_type      TEXT,
27          sha256        TEXT NOT NULL UNIQUE,
28          path          TEXT,
29          status        TEXT NOT NULL DEFAULT 'parsed',
30          created_at    TEXT NOT NULL DEFAULT (datetime('now'))
31      );
32
33      CREATE TABLE IF NOT EXISTS offers (
34          id          INTEGER PRIMARY KEY,
35          document_id INTEGER NOT NULL REFERENCES documents(id) ON DELETE CASCADE,
36          merchant    TEXT,
37          promo_code  TEXT,
38          description TEXT,
39          expiry_raw  TEXT,
40          expiry_type TEXT,
41          source      TEXT NOT NULL DEFAULT 'text',
42          redeemed    INTEGER NOT NULL DEFAULT 0,
43          source_text TEXT
44      );
45
46      CREATE INDEX IF NOT EXISTS idx_offers_document ON offers(document_id);
47
48      -- We use separate tables for each bank (e.g. axis_transactions, hdfc_transactions)
49      -- instead of a unified table to preserve raw schema fidelity of the source PDFs.
50      -- Normalization into a unified view layer is planned for a later phase
51      -- once all bank schemas are stabilized.
52      CREATE TABLE IF NOT EXISTS axis_transactions (
53          id INTEGER PRIMARY KEY,
54          document_id INTEGER NOT NULL REFERENCES documents(id) ON DELETE CASCADE,
55          account_number TEXT,
56          txn_date TEXT,
57          value_date TEXT,
```

```
        particulars TEXT,
        debit REAL,
        credit REAL,
        balance REAL
    );

    CREATE TABLE IF NOT EXISTS axis_cc_transactions (
        id INTEGER PRIMARY KEY,
        document_id INTEGER NOT NULL REFERENCES documents(id) ON DELETE CASCADE,
        account_number TEXT,
        txn_date TEXT,
        particulars TEXT,
        merchant_category TEXT,
        amount REAL,
        type TEXT
    );

    CREATE TABLE IF NOT EXISTS au_transactions (
        id INTEGER PRIMARY KEY,
        document_id INTEGER NOT NULL REFERENCES documents(id) ON DELETE CASCADE,
        account_number TEXT,
        txn_date TEXT,
        value_date TEXT,
        particulars TEXT,
        debit REAL,
        credit REAL,
        balance REAL
    );

    CREATE TABLE IF NOT EXISTS hdfc_transactions (
        id INTEGER PRIMARY KEY,
        document_id INTEGER NOT NULL REFERENCES documents(id) ON DELETE CASCADE,
        account_number TEXT,
        txn_date TEXT,
        value_date TEXT,
        particulars TEXT,
        ref TEXT,
        withdrawal REAL,
        deposit REAL,
        closing_balance REAL
    );
    """


    @contextmanager
    def connect(db_path: Path | str) -> Iterator[sqlite3.Connection]:
        """Open a connection (commit on success, always close). Parent dirs created."""
        path = str(db_path)
        if path != ":memory:":
            Path(path).parent.mkdir(parents=True, exist_ok=True)
        conn = sqlite3.connect(path)
        conn.row_factory = sqlite3.Row
        conn.execute("PRAGMA foreign_keys = ON")
        try:
            yield conn
            conn.commit()
        finally:
            conn.close()


    def init_db(conn: sqlite3.Connection) -> None:
        """Create tables if absent and stamp the schema version."""
        conn.executescript(_SCHEMA)
        conn.execute(f"PRAGMA user_version = {SCHEMA_VERSION}")

```

```

    def get_document_by_sha(conn: sqlite3.Connection, sha256: str) -> sqlite3.Row | None:
        return conn.execute("SELECT * FROM documents WHERE sha256 = ?",
(sha256,)).fetchone()


    def _insert_row(conn: sqlite3.Connection, table: str, fields: dict) -> int:
        """Internal helper for the simple INSERT pattern used by documents, offers, and bank
txns."""
        cols = ", ".join(fields)
        placeholders = ", ".join("?" for _ in fields)
        cur = conn.execute(
            f"INSERT INTO {table} ({cols}) VALUES ({placeholders})",
            tuple(fields.values()),
        )
        return int(cur.lastrowid)


    def insert_document(conn: sqlite3.Connection, **fields: object) -> int:
        """Insert into documents table. Delegates to the shared helper for DRY."""
        return _insert_row(conn, "documents", dict(fields))


    def insert_offer(conn: sqlite3.Connection, **fields: object) -> int:
        return _insert_row(conn, "offers", fields)


    def list_offers(conn: sqlite3.Connection) -> list[sqlite3.Row]:
        return conn.execute(
            """
            SELECT o.id, o.merchant, o.promo_code, o.expiry_raw, o.description, o.source,
                   d.external_id, d.path
            FROM offers o
            JOIN documents d ON d.id = o.document_id
            ORDER BY o.id
            """
        ).fetchall()


    def insert_axis_transaction(conn: sqlite3.Connection, **fields: object) -> int:
        return _insert_row(conn, "axis_transactions", fields)


    def insert_axis_cc_transaction(conn: sqlite3.Connection, **fields: object) -> int:
        return _insert_row(conn, "axis_cc_transactions", fields)


    def list_axis_cc_transactions(conn: sqlite3.Connection, limit: int = 50) ->
list[sqlite3.Row]:
        return conn.execute(
            "SELECT * FROM axis_cc_transactions ORDER BY id DESC LIMIT ?", (limit,)
        ).fetchall()


    def list_axis_transactions(conn: sqlite3.Connection, limit: int = 50) ->
list[sqlite3.Row]:
        return conn.execute(
            "SELECT * FROM axis_transactions ORDER BY id DESC LIMIT ?", (limit,)
        ).fetchall()


    def insert_hdfc_transaction(conn: sqlite3.Connection, **fields: object) -> int:
        return _insert_row(conn, "hdfc_transactions", fields)


```

```
    def list_hdfc_transactions(conn: sqlite3.Connection, limit: int = 50) ->
list[sqlite3.Row]:
        return conn.execute(
            "SELECT * FROM hdfc_transactions ORDER BY id DESC LIMIT ?", (limit,)
        ).fetchall()


    def insert_au_transaction(conn: sqlite3.Connection, **fields: object) -> int:
        return _insert_row(conn, "au_transactions", fields)


    def list_au_transactions(conn: sqlite3.Connection, limit: int = 50) ->
list[sqlite3.Row]:
        return conn.execute(
            "SELECT * FROM au_transactions ORDER BY id DESC LIMIT ?", (limit,)
        ).fetchall()

```

12. user (2026-05-31T14:28:02.112Z)

```
      """Secret storage via the OS keychain.

      OAuth tokens and cached statement passwords are stored here (Windows Credential
      Manager / macOS Keychain / Secret Service), never in the repo, on disk in
      plaintext, or in logs. See DESIGN.md section 5.

      This is the thin foundation used later by ingestion (token) and unlocking
      (per-sender cached password) ? see docs/ROADMAP.md milestones B.1 and C.1.
      """

      from __future__ import annotations

      import keyring
      from keyring.errors import PasswordDeleteError

      _SERVICE = "muneem"


      def get_secret(name: str) -> str | None:
          """Return the stored secret, or None if absent."""
          return keyring.get_password(_SERVICE, name)


      def set_secret(name: str, value: str) -> None:
          """Store (or replace) a secret in the OS keychain."""
          keyring.set_password(_SERVICE, name, value)


      def delete_secret(name: str) -> None:
          """Delete a secret if present; a no-op if it is not."""
          try:
              keyring.delete_password(_SERVICE, name)
          except PasswordDeleteError:
              pass

```

13. user (2026-05-31T14:28:02.194Z)

```
      """Local paths and configuration.

      muneem is local-first: config lives in the OS config dir, data (DB + staging)
      in the OS data dir. Nothing here touches the repository or the network, and no
      secrets are stored here ? those go through :mod:`muneem.secrets` (the keychain).
      """

```

```
8       from __future__ import annotations
9
10      import tomllib
11      from dataclasses import dataclass, field
12      from pathlib import Path
13      from typing import Any
14
15      from platformdirs import PlatformDirs
16
17      _dirs = PlatformDirs(appname="muneem", appauthor=False)
18
19      CONFIG_DIR = Path(_dirs.user_config_dir)
20      DATA_DIR = Path(_dirs.user_data_dir)
21      CONFIG_FILE = CONFIG_DIR / "config.toml"
22      DB_PATH = DATA_DIR / "muneem.db"
23      STAGING_DIR = DATA_DIR / "staging"
24
25
26      @dataclass(frozen=True)
27      class Config:
28          """Resolved local paths plus any values from ``config.toml``."""
29
30          config_dir: Path = CONFIG_DIR
31          data_dir: Path = DATA_DIR
32          config_file: Path = CONFIG_FILE
33          db_path: Path = DB_PATH
34          staging_dir: Path = STAGING_DIR
35          values: dict[str, Any] = field(default_factory=dict)
36
37
38      def load_config() -> Config:
39          """Load configuration, reading ``config.toml`` if it exists. Read-only."""
40          values: dict[str, Any] = {}
41          if CONFIG_FILE.is_file():
42              with CONFIG_FILE.open("rb") as fh:
43                  values = tomllib.load(fh)
44          return Config(values=values)
45
46
47      def ensure_dirs() -> None:
48          """Create the local config/data/staging dirs if missing. Never touches the repo."""
49          for directory in (CONFIG_DIR, DATA_DIR, STAGING_DIR):
50              directory.mkdir(parents=True, exist_ok=True)
51
```

14. assistant (2026-05-31T14:28:04.827Z)

Excellent. Now I have the context. Let me conduct deep research on encryption approaches for this threat model. I'll use the deep-research skill to gather authoritative sources on database encryption, OS-level encryption, key management, and threat models.

15. user (2026-05-31T14:28:07.233Z)

Launching skill: deep-research

16. user (2026-05-31T14:28:07.231Z)

Run the "deep-research" workflow.

Deep research harness ? fan-out web searches, fetch sources, adversarially verify claims, synthesize a cited report.

When the user wants a deep, multi-source, fact-checked research report on any topic. BEFORE invoking, check if the question is specific enough to research directly ? if underspecified

(e.g., "what car to buy" without budget/use-case/region), ask 2-3 clarifying questions to narrow scope. Then pass the refined question as args, weaving the answers in.

Phases:
- Scope: Decompose question (from args) into 5 search angles
- Search: 5 parallel WebSearch agents, one per angle
- Fetch: URL-dedup, fetch top 15 sources, extract falsifiable claims
- Verify: 3-vote adversarial verification per claim (need 2/3 refutes to kill)
- Synthesize: Merge semantic dupes, rank by confidence, cite sources

Invoke: Workflow({ name: "deep-research", args: "What is the threat model and best practice for at-rest encryption of a local SQLite database containing personal financial records on a single-user machine (Windows/Linux), considering: (a) OS full-disk encryption (BitLocker/LUKS) vs database-level encryption; (b) key storage in OS keychain vs passphrase-derived keys; (c) attack scenarios (stolen/discarded disk, synced backups, malware running as user, shoulder-surfing); (d) tools like sqlcipher, sqlite-fts5-encrypt, or OS-level solutions; (e) NIST/OWASP standards for local key management and encryption strategies? Focus on which approach adds real security value and which is redundant." })

17. assistant (2026-05-31T14:28:09.324Z)

I'll wait for the deep-research workflow to complete and compile the threat model analysis based on the findings.

18. user (2026-05-31T14:30:09.792Z)

Command running in background with ID: b1krnkwzf. Output is being written to: C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-muneem\a9fd4426-f234-4e4a-bfb4-1f2c8255c661\tasks\b1krnkwzf.output. You will be notified when it completes. To check interim output, use Read on that file path.

19. assistant (2026-05-31T14:30:12.591Z)

Let me check the research output and continue while we wait:

20. user (2026-05-31T14:30:12.839Z)

<system-reminder>Warning: the file exists but is shorter than the provided offset (1). The file has 1 lines.</system-reminder>

21. assistant (2026-05-31T14:30:15.142Z)

The workflow is still initializing. Let me proceed with targeted web searches directly to build the threat model while the deeper research runs:

22. user (2026-05-31T14:30:23.376Z)

Web search results for query: "NIST SP 800-111 mobile device encryption local database recommendations"

Links: [{"title":"Guide to Storage Encryption Technologies for End User Devices | NIST","url":"https://www.nist.gov/publications/guide-storage-encryption-technologies-end-user-devices"},{"title":"(PDF) NIST Special Publication 800-111: Guide to Storage Encryption Technologies for End User Devices","url":"https://www.researchgate.net/publication/329973300_NIST_Special_Publication_800-111_Guide_to_Storage_Encryption_Technologies_for_End_User_Devices"},{"title":"NIST SP 800-124 Rev. 1, Guidelines for Managing the Security of Mobile Devices in the Enterprise","url":"https://csrc.nist.rip/publications/nistpubs/800-124-rev1/sp800_124_r1.epub"},{"title":"NIST SP 800-111 Guide to Storage Encryption Technologies for End User Devices: NiST SP 800-111: National Institute of Standards and Technology: 9781548206567: Amazon.com: Books","url":"https://www.amazon.com/800-111-Storage-Encryption-Technologies-Devices/dp/1548206563"},{"title":"NIST Special Publication (SP) 800-111, Guide to Storage Encryption Technologies for End User Devices","url":"https://csrc.nist.gov/pubs/sp/800/111/final"},{"title":"SP 800-111, Guide to

Storage Encryption Technologies for End User Devices |
CSRC","url":"https://csrc.nist.rip/publications/detail/sp/800-111/final"},{"title":"NIST SP
800-111 Guide to Storage Encryption Technologies for End User Devices","url":"https://books.goog
le.com/books/about/NIST_SP_800_111_Guide_to_Storage_Encrypt.html?id=H7dfswEACAAJ"},{"title":"Gui
delines for Managing the Security of Mobile Devices i","url":"https://csrc.nist.gov/csrc/media/p
ublications/sp/800-124/rev-1/final/documents/draft_sp800-124-rev1.pdf"},{"title":"Special
Publication 800-111 Guide to Storage Encryption","url":"https://www.hhs.gov/sites/default/files/
ocr/privacy/hipaa/administrative/securityrule/nist800111.pdf?language=es"}]

Based on the search results, here's what I found about NIST SP 800-111:

NIST SP 800-111 is titled "Guide to Storage Encryption Technologies for End User Devices" and was created November 15, 2007.

Scope and Purpose:

This publication explains the basics of storage encryption, which is the process of using encryption and authentication to restrict access to and use of stored information. Many threats against end user devices, such as desktop and laptop computers, smart phones, personal digital assistants, and removable media, could cause information stored on the devices to be accessed by unauthorized parties. To prevent such disclosures of information, the information needs to be secured.

Encryption Solutions:

This publication describes three types of solutions?full disk encryption, volume and virtual disk encryption, and file/folder encryption?and makes recommendations for implementing and using each type.

Mobile Device Context:

Section 4 of NIST SP 800-124 (Guidelines for Managing the Security of Mobile Devices in the Enterprise) is based on Section 4 of NIST SP 800-111, indicating that SP 800-111's guidance is relevant to mobile device security.

Unfortunately, the search results don't contain specific details about local database encryption recommendations. For those specific details, you may want to access the full publication directly from the NIST website at https://csrc.nist.gov/pubs/sp/800/111/final.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

23. user (2026-05-31T14:30:23.512Z)

Web search results for query: "SQLite encryption Python keyring OS keychain security"

Links: [{"title":"charles leifer | Encrypted SQLite Databases with Python and
SQLCipher","url":"https://charlesleifer.com/blog/encrypted-sqlite-databases-with-python-and-
sqlcipher/"},{"title":"Best practices for securing
SQLite","url":"https://blackhawk.sh/en/blog/best-practices-for-securing-
sqlite/"},{"title":"Basic Security Practices for SQLite: Safeguarding Your Data - DEV
Community","url":"https://dev.to/stephenc222/basic-security-practices-for-sqlite-safeguarding-
your-data-23lh"},{"title":"Securing Your SQLite Database: Best Practices and
Techniques","url":"https://www.sqliteforum.com/p/securing-your-sqlite-database-
best"},{"title":"SQLite Encryption and Secure Storage for Sensitive
Data","url":"https://www.sqliteforum.com/p/sqlite-encryption-and-secure-
storage"},{"title":"Spelunking In SQLite && macOS Keychains - Secret
Chest","url":"https://www.secretchest.io/post/spelunking-in-sqlite-macos-
keychains"},{"title":"How to Implement Encryption with
SQLCipher","url":"https://oneuptime.com/blog/post/2026-02-02-sqlcipher-encryption/view"}]

Here are the search results for SQLite encryption with Python, keyring, OS keychain, and security:

Key Findings

SQLCipher for Encryption:
SQLCipher is an open-source library that provides transparent 256-bit AES encryption for SQLite databases, and provides transparent encryption for SQLite databases with minimal code changes, with the key to success being proper key management?generating strong keys, storing them securely, and never hardcoding them in your application.

Python Implementation:
You can use pysqlcipher3 with Python to set up encrypted databases using PRAGMA key statements. You can use environment variables or look into a library like python-keyring to manage encryption keys securely.

Secure Key Storage:
Mobile apps should use platform-specific secure storage like Android's Keystore or iOS Keychain, while server-side applications should use environment variables, Azure Key Vault, AWS KMS, or HashiCorp Vault. The passphrase must come from a secrets manager or OS keystore (not in code).

macOS Keychain Integration:
Keychain provides a built-in feature on macOS, iOS, and iPadOS devices that securely stores passwords, account numbers, and other confidential information such as certificates and tokens, and Apple built their keychain tool on top of SQLite, as did most every third party password manager.

Best Practices:
Encrypt at rest with SQLCipher, keep keys in a secrets manager and avoid hard-coding, set owner-only permissions on database files, use parameterized queries to prevent SQL injection, and consider secure_delete for sensitive deletes.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

24. user (2026-05-31T14:30:23.778Z)

Web search results for query: "SQLCipher vs full-disk encryption threat model local database"

Links: [{"title":"How to Implement Encryption with SQLCipher","url":"https://oneuptime.com/blog/post/2026-02-02-sqlcipher-encryption/view"},{"title":"How SQLCipher Compares to Other SQLite Encryption ...","url":"https://www.zetetic.net/sqlcipher/comparison/"},{"title":"How to Encrypt Your Room Database in Android Using SQLCipher ? | by Pouya Heydari | ProAndroidDev","url":"https://proandroiddev.com/how-to-encrypt-your-room-database-in-android-using-sqlcipher-0bce78328bd6?gi=2e0392186b8c"},{"title":"open-mobile-developers-guide/en/4-security-and-privacy/encrypt-all-the-bits-introduction-to-mobile-security.md at master · guardianproject/open-mobile-developers-guide","url":"https://github.com/guardianproject/open-mobile-developers-guide/blob/master/en/4-security-and-privacy/encrypt-all-the-bits-introduction-to-mobile-security.md"},{"title":"SQLCipher: Encrypted Database | Guardian Project","url":"https://guardianproject.info/code/sqlcipher/"},{"title":"Selecting the Right Encryption Approach","url":"https://cpl.thalesgroup.com/encryption/selecting-right-encryption-approach"},{"title":"Popular SQL Server Database Encryption Choices","url":"https://arxiv.org/pdf/1901.03179"},{"title":"Transparent Data Encryption -- Solution for Security of Database Contents","url":"https://arxiv.org/pdf/1303.0418"},{"title":"Practical Cryptographic Data Integrity Protection with Full Disk Encryption Extended Version","url":"https://arxiv.org/pdf/1807.00309"}]

Based on the search results, here's what I found regarding SQLCipher versus full-disk encryption in the context of local database threat modeling:

SQLCipher's Approach

SQLCipher solves the problem of plain text database storage by providing transparent, full-

database encryption using AES-256. SQLCipher encrypts the entire database file, including metadata, tables, indexes, and journal files. Pages are encrypted before being written to disk and are decrypted when read back. Additionally, SQLCipher implements an additional security feature where each page's data is protected by an HMAC "signature", such that if any page data is modified externally or resequenced SQLCipher will detect the invalid change when the page is read back.

Full-Disk Encryption (FDE) Limitations

In contrast, full-disk encryption (FDE) and self-encrypting drives (SED) encrypt data as it is written to the disk and decrypt data as it is read off the disk. However, FDE addresses a very limited set of threats?protects only from physical loss of storage media and lacks safeguards against advanced persistent threats (APTs), malicious insiders, or external attackers.

Key Threat Model Difference

In general, the lower in the stack encryption is employed, the simpler and less intrusive the implementation will be. However, the number and types of threats these data encryption approaches can address are also reduced. By employing encryption higher in the stack, organizations can typically realize higher levels of security and mitigate more threats.

In essence, SQLCipher provides application-level database encryption with integrity checking, while FDE operates at the disk level with limited threat coverage against sophisticated attacks.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

25. user (2026-05-31T14:30:24.333Z)

Web search results for query: "OWASP local key management passphrase KDF Argon2 PBKDF2"

Links: [{"title":"Password Storage - OWASP Cheat Sheet Series","url":"https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html"},{"title":"Why Developers Should Stop Hashing Passwords and Start Using KDFs | by Felix Twoli | Medium","url":"https://medium.com/@felixtwoli21/why-developers-should-stop-hashing-passwords-and-start-using-kdfs-0be0da663393"},{"title":"Argon2: The Password Hashing Algorithm Behind AudioCipher Encryption | AudioCipher","url":"https://audiocipher.app/blog/argon2-key-derivation"},{"title":"Encryption Key Derivation | Bitwarden","url":"https://bitwarden.com/help/kdf-algorithms/"},{"title":"Best practices for password hashing and storage","url":"https://datatracker.ietf.org/doc/id/draft-whited-kitten-password-storage-00.html"},{"title":"The Evolution of Password Hashing | Psono","url":"https://psono.com/blog/evolution-of-password-hashing"},{"title":"PBKDF2 - Wikipedia","url":"https://en.wikipedia.org/wiki/PBKDF2"},{"title":"Password Storage · OWASP Cheat Sheet Series","url":"https://owasp.deteact.com/cheat/cheatsheets/Password_Storage_Cheat_Sheet.html"},{"title":"Williamdes's blog - Secure defaults for LDAP password (CRYPT/PBKDF2/SHA2/ARGON2) storage","url":"https://blog.williamdes.eu/Infrastructure/tutorials/secure-ldap-password-crypt-pbkdf2-sha2-argon2-defaults/"}]

I've completed a web search for your query about OWASP local key management, passphrase, KDF, Argon2, and PBKDF2. Here are the key findings:

OWASP Recommendations for Password/Passphrase Protection

Passwords should be securely hashed using modern, adaptive hashing algorithms (e.g., Argon2id, bcrypt, or PBKDF2), rather than encrypted or stored in plaintext.

Argon2id - Primary Recommendation

Argon2 was the winner of the 2015 Password Hashing Competition. Out of the three Argon2 versions, use the Argon2id variant since it provides a balanced approach to resisting both side-channel and GPU-based attacks.

Use Argon2id with a minimum configuration of 19 MiB of memory, an iteration count of 2, and 1 degree of parallelism.

PBKDF2 - FIPS-Compliant Alternative

If FIPS-140 compliance is required, use PBKDF2 with a work factor of 600,000 or more and set with an internal hash function of HMAC-SHA-256.

Key Derivation Function (KDF) Purpose

Key Derivation Functions (KDFs) solve password security by making the computation expensive. Deriving one key takes significant time and memory. Testing a million passphrases requires a million expensive computations ? making brute force attacks impractical.

Comparison of Algorithms

One weakness of PBKDF2 is that while its number of iterations can be adjusted to make it take an arbitrarily large amount of computing time, it can be implemented with a small circuit and very little RAM, which makes brute-force attacks using application-specific integrated circuits or graphics processing units relatively cheap.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

26. user (2026-05-31T14:30:25.282Z)

Web search results for query: "BitLocker LUKS full-disk encryption vs application-level encryption"

Links: [{"title":"Linux Disk Encryption with LUKS - zenarmor.com","url":"https://www.zenarmor.com/docs/linux-tutorials/linux-disk-encryption-with-luks"},{"title":"Disk Encryption Tools and Which Is Best For You | by Cybersecurity Stephen | Medium","url":"https://medium.com/@cybersecuritystephen/disk-encryption-tools-and-which-is-best-for-you-d300a8bbfe75"},{"title":"Data Encryption Practices: Comparing Linux, Windows, and macOS | by Aditya Bhuyan | Medium","url":"https://aditya-sunjava.medium.com/data-encryption-practices-comparing-linux-windows-and-macos-e8a6d4075ec8"},{"title":"LUKS: the Linux alternative to BitLocker for truly sovereign encryption ? Le Goût du Libre","url":"https://legoutdulibre.com/en/2026/01/26/luks-the-linux-alternative-to-bitlocker-for-truly-sovereign-encryption/"},{"title":"BitLocker Vs FileVault Vs LUKS Vs VeraCrypt: Enterprise Matrix - Newsoftwares.net Blog","url":"https://www.newsoftwares.net/blog/bitlocker-vs-filevault-vs-luks-vs-veracrypt-enterprise-matrix/"},{"title":"BitCracker: BitLocker meets GPUs","url":"https://arxiv.org/pdf/1901.01337"},{"title":"Key management in full disk and file-level encryption","url":"https://image-ppubs.uspto.gov/dirsearch-public/print/downloadPdf/9177160"},{"title":"System and method for encryption of disk based on pre-boot compatibility testing","url":"https://image-ppubs.uspto.gov/dirsearch-public/print/downloadPdf/9384353"}]

Based on the search results, here's a comparison of BitLocker, LUKS, and full-disk encryption versus application-level encryption:

Full-Disk Encryption (BitLocker and LUKS)

BitLocker:
BitLocker is Microsoft's full-disk encryption solution available on Windows 10 Pro, Enterprise, and Education editions, designed to provide easy-to-implement, transparent encryption integrating seamlessly with the operating system. BitLocker uses AES encryption to encrypt the entire drive and can be configured to require a PIN or password at startup, or work with hardware-based encryption via Trusted Platform Module (TPM) chips.

LUKS:
LUKS encrypts data at the block level and is often used alongside the dm-crypt subsystem to provide strong disk encryption. LUKS provides full-disk encryption, meaning every file, including system data, swap partitions, and temporary files, is covered.

Key Differences

Coverage and Protection:
With full-disk encryption, there are no weak points left behind, making it far harder for attackers to bypass encryption and extract private data. In contrast, file-level encryption allows for protection of data from entities (users, processes, etc.) even on the same system, for which full-disk encryption does not protect.

Limitations of LUKS:
LUKS provides full-disk or partition-level encryption, meaning you cannot encrypt individual files or folders.

Integration and Complexity:
BitLocker and FileVault are tightly integrated with their respective operating systems, providing seamless user experiences, while LUKS offers more flexibility and control, but at the cost of increased complexity.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

27. assistant (2026-05-31T14:30:27.322Z)

Excellent. Now let me search for more specific information on malware/runtime threats and keyring security:

28. user (2026-05-31T14:30:34.267Z)

Web search results for query: "keyring Python keyring module Windows Credential Manager DPAPI security"

Links: [{"title":"Windows Credential Manager ? e-lo:Engineer log out.","url":"https://follow-e-lo.com/2024/11/22/windows-credential-manager/"},{"title":"Python Keyring Backends: SecretService Windows Credential Manager Support 2025","url":"https://johal.in/python-keyring-backends-secretservice-windows-credential-manager-support-2025/"},{"title":"Password Management in Python ? Keyring and Credential Manager","url":"https://medium.com/@aarhar/password-management-in-python-keyring-and-credential-manager-29fa4ccc919e"},{"title":"Storing and Accessing Passwords in Python Using Keyring (Windows 10 and Linux) | by Jamie Southchurch | Medium","url":"https://jamie-southchurch.medium.com/storing-and-accessing-passwords-in-python-using-keyring-windows-10-and-linux-648cf07beef6"},{"title":"keyring · PyPI","url":"https://pypi.org/project/keyring/"},{"title":"Secure Password or Token on Windows ? Python | Tech Wizard","url":"https://techwizard.cloud/2022/02/09/secure-password-or-token-on-windows-python/"},{"title":"How to Configure Python Keyring with Windows Credential Manager on Windows 7 - YouTube","url":"https://www.youtube.com/watch?v=F_TGPFnIgjQ"},{"title":"keyring/keyring/backends/Windows.py at main · jaraco/keyring","url":"https://github.com/jaraco/keyring/blob/main/keyring/backends/Windows.py"}]

Here are the search results for your query about the Python keyring module, Windows Credential Manager, and DPAPI security:

Key Findings:

Integration Between Keyring and Windows Credential Manager:
The keyring library provides a cross-platform way to retrieve credentials, and on Windows, it integrates with Credential Manager. The Keyring package in Python is designed for storing encrypted passwords in your operating systems password vault (Window's Credential Manager, KWallet for KDE in Linux).

How Windows Credential Manager Uses DPAPI:
Windows Credential Manager leverages CredEnumerate/CredWrite APIs, tying encryption to user DPAPI (Data Protection API) blobs, inherently scoped to user SID for isolation. DPAPI encryption is tied to the user's account credentials.

Basic Usage:

You can store passwords in the keyring using `set_password(service, username, password)` and delete them using `delete_password(service, username)`.

Security Considerations:
Credential Manager doesn't allow fine-grained access control to restrict which applications can access specific credentials. If a malicious application runs in the user's context, it could potentially abuse the stored credentials. Additionally, by default, Windows Credential Manager does not provide detailed logs of when and how credentials are accessed, which can make it challenging to detect unauthorized access.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

29. user (2026-05-31T14:30:34.425Z)

Web search results for query: "SQLCipher license MIT BSD Apache open source"

Links: [{"title":"SQLCipher License - Commercial and Community License Information and Notices | Zetetic","url":"https://www.zetetic.net/sqlcipher/license/"},{"title":"Applicenses Archive - uMotif","url":"https://umotif.com/app-licenses/trialmotif-app-licenses/"},{"title":"SQLCipher: Encrypted Database | Guardian Project","url":"https://guardianproject.info/code/sqlcipher/"},{"title":"SQLCipher and Licensing | B4X Programming Forum","url":"https://www.b4x.com/android/forum/threads/sqlcipher-and-licensing.50676/"},{"title":"SQLCipher · GitHub","url":"https://github.com/sqlcipher"},{"title":"sqlcipher/sqlcipher | DeepWiki","url":"https://deepwiki.com/sqlcipher/sqlcipher"},{"title":"GitHub - sqlcipher/sqlcipher: SQLCipher is a standalone fork of SQLite that adds 256 bit AES encryption of database files and other security features. · GitHub","url":"https://github.com/sqlcipher/sqlcipher"},{"title":"sqlcipher/SQLITE_LICENSE.md at master · sqlcipher/sqlcipher","url":"https://github.com/sqlcipher/sqlcipher/blob/master/SQLITE_LICENSE.md"},{"title":"sqlcipher/LICENSE.txt at master · sqlcipher/sqlcipher","url":"https://github.com/sqlcipher/sqlcipher/blob/master/LICENSE.txt"}]

Based on the search results, here's what I found about SQLCipher's licensing:

SQLCipher License

SQLCipher code itself is licensed under a BSD-style license from Zetetic LLC. More specifically, the project is developed by Zetetic LLC and distributed under a BSD 3-Clause license.

Community Edition

The Community Edition is available under a BSD-style license and requires attribution and reproduction of license grants in the application interface and/or materials. Additionally, SQLCipher Community Edition can be used in both open source and closed source commercial software, but you must include the BSD-style license and copyright statements prominently in the application and documentation.

Additional Licensing Information

The search results show that while SQLCipher itself uses a BSD-style license, it includes various dependencies with different licenses:
- OpenSSL is distributed under the Apache License v2 or dual OpenSSL and SSLeay License
- SQLite JDBC is distributed under the MIT license and BSD License
- SQLite is released to the Public Domain

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

30. user (2026-05-31T14:30:34.778Z)

Web search results for query: "malware running as user privileges FDE SQLCipher threat model"

Links: [{"title":"Protecting SQL Server from Ransomware","url":"https://www.mssqltips.com/sqlservertip/5054/protecting-sql-server-from-ransomware/"},{"title":"What is Privilege Escalation? Attacks & Defense Guide | BeyondTrust","url":"https://www.beyondtrust.com/blog/entry/privilege-escalation-attack-defense-explained"},{"title":"System and method for zero-day privilege escalation malware detection","url":"https://image-ppubs.uspto.gov/dirsearch-public/print/downloadPdf/9959406"},{"title":"Applying fractional calculus to malware spread: A fractal-based approach to threat analysis","url":"https://www.ncbi.nlm.nih.gov/pmc/articles/PMC11709322/"},{"title":"Threat Actors Exploiting MS-SQL Servers to Deploy XiebroC2 Framework","url":"https://gbhackers.com/ms-sql-servers/"},{"title":"Practical Enclave Malware with Intel SGX","url":"https://arxiv.org/pdf/1902.03256"}]

Based on the search results, I found some relevant information about malware running with user privileges, though the results don't specifically address SQLCipher and FDE threat models together.

Here's what the search results reveal:

Malware Operating with User Privileges:

Most malware will execute as the user, with the rights and privileges of said user, and this is especially true of ransomware. While some malware will attempt to take advantage of other vulnerabilities to escalate privileges, this tends to work only on the local system, not remotely, unless it's a particularly complex piece of malware.

The search results also include information about privilege escalation techniques that malware may employ. For example, threat actors can use tools like JuicyPotato, a privilege-escalation utility exploiting specific Windows privileges within the tokens of running processes, to elevate from low-privilege accounts to SYSTEM privileges.

However, the search results don't contain specific information addressing SQLCipher encryption or Full Disk Encryption (FDE) threat models in relation to malware running as user privileges. To get more targeted information on those specific topics, you might want to search for security documentation on SQLCipher threat models or FDE limitations against user-level malware separately.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

31. user (2026-05-31T14:30:35.354Z)

Web search results for query: "encrypted database query performance WHERE aggregation SQL"

Links: [{"title":"Efficient Execution of Aggregation Queries over Encrypted Relational Databases | Request PDF","url":"https://www.researchgate.net/publication/220788001_Efficient_Execution_of_Aggregation_Queries_over_Encrypted_Relational_Databases"},{"title":"E?cient Execution of Aggregation Queries over Encrypted Relational Databases","url":"https://www.comp.nus.edu.sg/~tankl/cs5322/readings/dasfaa04.pdf"},{"title":"On query execution over encrypted data","url":"https://www.academia.edu/10034523/On_query_execution_over_encrypted_data"},{"title":"SQL Server Column Level Encryption and Query Performance - SQLRx","url":"https://www.sqlrx.com/sql-server-column-level-encryption-and-query-performance/"},{"title":"Full article: SQL queries over encrypted databases: a survey","url":"https://www.tandfonline.com/doi/full/10.1080/09540091.2024.2323059"},{"title":"Query Processing Performance and Searching Over Encrypted Data By Using An Efficient Algorithm","url":"https://arxiv.org/pdf/1308.4687"},{"title":"CryptDB: Processing Queries on an Encrypted Database","url":"https://crypto.stanford.edu/RealWorldCrypto/slides/raluca.pdf"},{"title":"System and method for fast querying of encrypted databases","url":"https://image-ppubs.uspto.gov/dirsearch-public/print/downloadPdf/7395437"},{"title":"System and method for fast querying of encrypted databases","url":"https://image-ppubs.uspto.gov/dirsearch-public/print/downloadPdf/7890774"}]

I've found several relevant sources on encrypted database query performance with WHERE clauses and aggregation operations. Here's what the search results reveal:

Key Findings

Query Performance Challenges:
It is very difficult to process encrypted data without having to decrypt it, though research has developed techniques to address this problem.

Aggregation Operations:
Systems can perform equality and range queries, the aggregation operations of MAX, MIN, and COUNT, as well as GROUPBY and ORDERBY operations, directly on encrypted data without first decrypting the operands.

Performance Improvements:
Using searchable columns in WHERE clauses can significantly improve response time, from 3 minutes to a little over 320 ms on a 10 million record table.

Encryption Approaches:
Research proposes generating unique hash values for sensitive data and translating SQL clauses into appropriate forms to execute over hash value attributes to improve query performance on encrypted databases.

Modern Solutions:
Hardware-based databases using trusted execution environments can execute SQL queries directly on plaintext within trusted hardware, supporting complex SQL functionalities and computations such as joins, aggregations, order by, and group by.

The search results include academic papers, patent documents, and technical articles discussing various approaches to balancing data encryption with query processing efficiency.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

32. assistant (2026-05-31T14:30:40.187Z)

Perfect. Now let me search for information about specific Python bindings and Linux/Windows deployment scenarios:

33. user (2026-05-31T14:30:47.158Z)

Web search results for query: "pysqlcipher3 python-sqlcipher wheels prebuilt binaries Windows Linux"

Links: [{"title":"sqlcipher3-wheels · PyPI","url":"https://pypi.org/project/sqlcipher3-wheels/"},{"title":"GitHub - rotki/pysqlcipher3: Configuration to build and publish pysqlcipher3 with wheels for rotki","url":"https://github.com/rotki/pysqlcipher3"},{"title":"sqlcipher3 · PyPI","url":"https://pypi.org/project/sqlcipher3/"},{"title":"pysqlcipher3 · PyPI","url":"https://pypi.org/project/pysqlcipher3/"},{"title":"GitHub - coleifer/sqlcipher3: Python 3 bindings for SQLCipher · GitHub","url":"https://github.com/coleifer/sqlcipher3"},{"title":"sqlcipher3-wheels 0.5.7 on PyPI - Libraries.io - security & maintenance data for open source software","url":"https://libraries.io/pypi/sqlcipher3-wheels"},{"title":"rotki-pysqlcipher3 · PyPI","url":"https://pypi.org/project/rotki-pysqlcipher3/"},{"title":"pysqlcipher3-binary · PyPI","url":"https://pypi.org/project/pysqlcipher3-binary/"}]

Based on the search results, here's what I found regarding pysqlcipher3, python-sqlcipher wheels, and prebuilt binaries for Windows and Linux:

Available Packages with Wheels

A completely self-contained binary package (wheel) is available starting with version 0.6.2,

which contains the latest release of SQLCipher compiled with numerous extensions, and requires no external dependencies.

Main Options:

1. **sqlcipher3** - A completely self-contained binary package (wheel) is available for versions 0.4.0 and newer as sqlcipher3-binary, containing the latest release of sqlcipher compiled with numerous extensions, and requiring no external dependencies.

2. **sqlcipher3-wheels** - This is a fork of sqlcipher3 which adds github action for creating wheels for Windows, MacOS and Linux, with unofficial wheels from this fork uploaded to sqlcipher3-wheels on pypi, while the official wheels are sqlcipher3-binary.

3. **rotki-pysqlcipher3** - This build has SQLCipher 4.x statically linked.

4. **pysqlcipher3-binary** - An older option for prebuilt binaries.

Platform Support

The available wheel distributions include support for:
- Windows (win_amd64, win32, win_arm64)
- Linux (manylinux, musllinux variants for x86_64, aarch64, i686)
- macOS

All of these options provide prebuilt binary wheels that eliminate the need to compile SQLCipher and its dependencies from source.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

34. user (2026-05-31T14:30:47.504Z)

Web search results for query: ""cryptography" Python library license Fernet encryption wheels"

Links: [{"title":"cryptography · PyPI","url":"https://pypi.org/project/cryptography/"},{"title":"cryptography/src/cryptography/fernet.py at main · pyca/cryptography","url":"https://github.com/pyca/cryptography/blob/main/src/cryptography/fernet.py"},{"title":"FernetEncryption · PyPI","url":"https://pypi.org/project/FernetEncryption/"},{"title":"cryptography Python (How It Works For Developers)","url":"https://ironpdf.com/python/blog/python-help/cryptography-python/"},{"title":"python-cryptography-fernet-wrapper · PyPI","url":"https://pypi.org/project/python-cryptography-fernet-wrapper/"},{"title":"Fernet (symmetric encryption) ? Cryptography 49.0.0.dev1 documentation","url":"https://cryptography.io/en/latest/fernet/"},{"title":"Welcome to pyca/cryptography ? Cryptography 49.0.0.dev1 documentation","url":"https://cryptography.io/"},{"title":"Cryptography with Python using Fernet - DEV Community","url":"https://dev.to/anishde12020/cryptography-with-python-using-fernet-40o3"},{"title":"Encryption and Decryption with Fernet in Python | by Rahul Gosavi | Medium","url":"https://medium.com/@rahulgosavi.94/fernet-encryption-and-decryption-2101f0e3097a"}]

Based on the web search results, here's the information about the cryptography Python library:

Overview

The cryptography library is licensed under Apache-2.0 OR BSD-3-Clause, making it a dual-licensed project. It is maintained by the Python Cryptographic Authority.

Fernet Encryption

The library includes a high-level symmetric encryption recipe through its Fernet module, which allows you to encrypt messages by generating a key and using it to encrypt data. The result of Fernet encryption is known as a "Fernet token" and has strong privacy and authenticity guarantees.

Wheels and Distribution

The cryptography library is distributed through PyPI with multiple wheel files available for different Python versions and platforms, including versions for CPython 3.9+, CPython 3.11+, and CPython 3.14, across platforms like manylinux, macOS, and Windows.

Library Capabilities

The cryptography library includes both high-level recipes and low-level interfaces to common cryptographic algorithms such as symmetric ciphers, message digests, and key derivation functions.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

35. user (2026-05-31T14:30:48.929Z)

Web search results for query: "defense in depth encryption local database backups synced cloud"

Links: [{"title":"Defense-in-Depth for Cloud Databases","url":"https://www.oracle.com/technetwork/database/security/wp-security-dbsec-cloud-3225125.pdf"},{"title":"Defense in Depth: Combining Persistent and Transparent Data Encryption - Security Boulevard","url":"https://securityboulevard.com/2026/03/defense-in-depth-combining-persistent-and-transparent-data-encryption/"},{"title":"SQL Server Encrypted Backups: Transparent Data Encryption or Backup Encryption ? Learn SQL Server High Availability & Disaster Recovery","url":"https://learnsqlserverhadr.com/sql-server-encrypted-backups-transparent-data-encryption-or-backup-encryption/"},{"title":"Ransomware Defense In-Depth: Rubrik Fully-Encrypted Lifecycle Management | Rubrik","url":"https://www.rubrik.com/blog/technology/2021/08/rubrik-fully-encrypted-lifecycle-management"},{"title":"How to Build a Defense-in-Depth Strategy for Your Database Infrastructure","url":"https://www.navicat.com/en/company/aboutus/blog/3845-how-to-build-a-defense-in-depth-strategy-for-your-database-infrastructure.html"},{"title":"Data Encryption at Rest, Transparent Data Encryption, and BitLocker - Business Central | Microsoft Learn","url":"https://learn.microsoft.com/en-us/dynamics365/business-central/dev-itpro/security/transparent-data-encryption"},{"title":"Azure data encryption at rest - Azure Security | Microsoft Learn","url":"https://learn.microsoft.com/en-us/azure/security/fundamentals/encryption-atrest"}]

I've completed the web search for your query. Here's a summary of the key findings:

Defense-in-Depth Encryption Strategy

Transparent Data Encryption (TDE) provides efficient, block-level encryption for critical systems and databases while ensuring minimal operational overhead and protecting data at rest in known locations.

TDE protects data "on disk" by encrypting databases, files, and backups at the storage or file system level. However, as sensitive data moves to cloud services, partners, and remote users, it's essential to protect the data itself with Persistent Data Encryption (PDE).

Encryption for Backups and Cloud Sync

Data at rest ? the actual files stored on disk, including backups ? should be encrypted so that physical access to storage media does not translate into data exposure. Encryption should be applied throughout the entire backup and recovery process, ensuring that data both at rest and in-flight is encrypted and safe from ransomware and other malicious attacks, whether data is local or within the cloud.

Layered Approach

A defense-in-depth approach requires layers upon layers of protection to minimize security breaches. Data in transit ? the queries, results, and credentials that travel between clients and database servers ? should always be encrypted using Transport Layer Security (TLS).

The search results emphasize that combining multiple encryption technologies creates a more

robust security posture for database infrastructure and cloud backups.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

36. assistant (2026-05-31T14:31:49.180Z)

Excellent research data. Now let me compile this into a comprehensive threat model report. Let me create the final analysis:

THREAT MODEL & ENCRYPTION ARCHITECTURE FOR MUNEEM PERSONAL-FINANCE DATABASE

Executive Summary

For muneem's single-user, local-first personal-finance SQLite database on Windows/Linux, **database-level encryption (SQLCipher) layered on top of OS full-disk encryption (BitLocker/LUKS) is the recommended defense-in-depth approach**. However, the marginal security gain from database encryption against certain threat categories (stolen/discarded disk) is moderate; the dominant value lies in protecting against **malware running as the user** and **cloud backup scenarios**. This analysis provides a transparent threat model addressing each scenario.

PART A: THREAT SCENARIOS & WHAT EACH ENCRYPTION LAYER STOPS

Scenario 1: Physical Disk Theft / Hardware Discarded

Threat: Attacker has physical access to the storage media; user is offline.

Scenario	FDE Only (BitLocker/LUKS)	FDE + SQLCipher	Notes
Stolen drive, attacked offline	? Stops (if TPM+PIN or strong passphrase)	?? Stops (defense-in-depth)	FDE is sufficient for this threat. SQLCipher adds zero marginal value if the key is in the OS keychain on the same disk?the attacker extracts the keychain entry and unlocks the database.
Stolen drive, raw sector read	? Stops (encryption at block level)	? Stops (page-level encryption)	Both provide full coverage.

Verdict: FDE alone is **adequate** for stolen/discarded hardware. SQLCipher does NOT add real protection here because:
- The encryption key (if stored in OS keychain via Windows Credential Manager or Linux Secret Service) is decryptable via the same user account that's registered to the disk.
- Passphrase-derived keys stored in keychain would require the user to re-enter the passphrase, but if an attacker has shell access, they can still extract keychain material.

Scenario 2: Cloud Backup Sync (Backblaze, OneDrive, Google Drive, etc.)

Threat: Database file is synced to cloud storage; attacker breaches cloud provider or account.

Layer	FDE Only	FDE + SQLCipher	Notes
Local copy	? Encrypted at rest on disk	?? Double-encrypted	FDE protects the local copy.
Cloud copy	? **NOT encrypted at rest in cloud**	? Cloud sees encrypted blob	FDE applies only to local storage. Cloud provider stores plaintext (or applies their own encryption, which is opaque). **SQLCipher is essential here.**
In-transit to cloud	Depends on sync client (usually TLS)	Same (TLS covers network)	SQLCipher doesn't affect network encryption; TLS does.

Verdict: SQLCipher is **critical** if backups are synced to uncontrolled cloud storage. FDE alone is **insufficient** for this threat.

Scenario 3: Malware Running as User

Threat: Ransomware, spyware, or trojan executes in user context (no privilege escalation).

Layer	FDE Only	FDE + SQLCipher	Notes
Malware reads plaintext DB from disk	? Encrypted at block level, not visible	? **Encrypted file, but key is in keychain**	FDE is effective *if* the disk is not mounted or the malware can't bypass volume lock. But if the OS is running, the FDE volume is **already decrypted** in RAM (TPM unlock, user logged in). Malware running as user can read plaintext pages directly from `/dev/shm` or `/mnt` or memory.
Malware accesses keychain/Credential Manager	N/A	? **Key extracted; DB decrypted**	Windows Credential Manager and Linux Secret Service (KDE Wallet, GNOME Keyring) store keys in user-accessible vaults. Malware running as user **can retrieve the key**.
Malware reads memory (unencrypted database pages)	N/A	? Unencrypted pages in memory	SQLCipher decrypts pages into RAM for queries. Malware can dump process memory or scan for plaintext patterns.

Verdict: **Neither FDE nor SQLCipher provides strong defense** against user-level malware. The mitigations are:
1. **Passphrase-protected database** (user types it in; key NOT stored in keychain)?but this breaks the single-user app model.
2. **OS-level defense**: EDR (endpoint detection), privilege escalation prevention, sandboxing.
3. **SQLCipher buys time**: Attacker must extract the key from keychain or dump memory; requires more sophistication than plaintext read.

Scenario 4: Shoulder Surfing / Physical Spy

Threat: Attacker watches user unlock DB or reads screen.

Layer	Protection
FDE	? Passphrase at boot is hidden by `*` entry.
SQLCipher + Keyring	? Key fetched silently; user doesn't type DB passphrase (reduces exposure window).
SQLCipher + Passphrase in app	? User sees prompt on screen; attacker may see input.

Verdict: Keyring-based approach (FDE + SQLCipher + keyring-stored key) is **better than passphrase-entry** for shoulder-surf defense.

PART B: KEY MANAGEMENT OPTIONS & THEIR SECURITY PROFILE

Option 1: Random Key in OS Keychain (Current muneem approach for OAuth tokens)

How it works:
- App calls `keyring.set_password("muneem", "db_encryption_key", <strong>random_bytes(32)</strong>)`
- OS stores in Credential Manager (Windows DPAPI) or Secret Service (Linux)
- On first use, app retrieves key and opens database

Security profile:
- ? Key never in source code or disk file
- ? Windows: DPAPI ties encryption to user account SID; extraction requires user-level process
- ? Linux: Secret Service (org.freedesktop.Secret.Service) requires dbus user session; attackers must hijack session
- ? **Malware in user context CAN extract key** (see Scenario 3)
- ? **If user's account is compromised** (weak password, phishing), attacker can retrieve key

Verdict: Appropriate for **single-user, local-first** model. Defends against casual attackers and disk theft. Insufficient against sophisticated user-level malware.

Option 2: Passphrase + KDF (Argon2id or PBKDF2)

How it works:
- User provides passphrase (or config file with passphrase?still a security issue)
- App derives key: `Argon2id(passphrase, salt, params) ? 256-bit key`
- Key is ephemeral (not stored); database locked when app closes

Security profile:
- ? No key stored anywhere; stronger against malware harvest
- ? Workload (memory + iterations) makes brute-force expensive
- ? If database file is stolen, attacker must brute-force passphrase (not extract it)
- ? Passphrase entropy depends on user choice (typically weak)
- ? **Passphrase in config file negates all benefits** (attacker reads plaintext file)
- ? Breaks "single-user local-first" model if user must type passphrase repeatedly

Verdict: **Better than random key for offline dictionary attacks**, but **incompatible with muneem's frictionless local-first design**. Suitable for:
- Multi-user shared machines
- High-threat scenarios (e.g., financial advisor using shared workstation)
- Offline backup encryption (encrypt DB at rest before shipping to external drive)

Option 3: Hybrid (Random Key in Keychain + Passphrase Option)

How it works:
- Default: app uses keyring-stored random key (frictionless)
- Optional: user can set a master passphrase that unlocks the keychain secret or protects the key derivation

Security profile:
- ? Best of both: convenience + optional hardening
- ? Passphrase adds friction against malware that auto-extracts keychain
- ? Adds complexity
- ? Passphrase still weak if user-chosen

Verdict: **Reasonable middle ground** if threat model demands higher assurance. Not required for baseline defense.

PART C: OS KEYCHAIN / SECRET MANAGEMENT

Windows: Credential Manager & DPAPI

- **Backend**: Windows Credential Manager via DPAPI (Data Protection API)
- **Scope**: Tied to user account SID
- **Threat model**: Protects against:
  - ? Physical disk theft (key encrypted with derived key from user password)
  - ? Other user accounts on same machine
  - ? **Same user account** (malware in user context can call DPAPI decrypt)
- **Python access**: `keyring` library (version 25.0+) delegates to Windows Credential Manager
- **License**: Windows Credential Manager is OS-provided (no license concern)

Reference: [Python Keyring Backends: SecretService Windows Credential Manager Support 2025](https://johal.in/python-keyring-backends-secretservice-windows-credential-manager-support-2025/)

Linux: Secret Service (org.freedesktop.Secret.Service)

- **Backend**: GNOME Keyring / KDE Wallet / pass / secretstorage
- **Scope**: User session; stored in `~/.local/share/keyrings/` (encrypted by login session)
- **Threat model**: Similar to Windows (user-level access can extract)

- **Python access**: `keyring` library auto-detects backend
- **License**: Open source (LGPL, MIT, etc. depending on backend)

Key limitation across both platforms: If an attacker gains code execution in the user's context, the keyring CAN be accessed. This is by design?keyrings are not meant to protect against user-level malware, only against other users and physical theft.

PART D: SQLCIPHER vs. ENCRYPTION APPROACHES

Option A: SQLCipher (Full-Database AES-256 Encryption)

What it does:
- Encrypts entire SQLite file at the page level (4 KB pages)
- Each page encrypted with AES-256 in CBC mode + HMAC integrity check
- Transparent to SQL queries (WHERE, aggregates work without decryption changes)
- Key provided via `PRAGMA key = 'passphrase'` or hex string

Advantages:
- ? Transparent: Existing SQL queries unchanged
- ? Integrity: HMAC detects tampering
- ? Can be used with keyring-provided key: `conn.execute(f"PRAGMA key = '{key_from_keyring}';")`
- ? Prebuilt wheels available (sqlcipher3-wheels, rotki-pysqlcipher3, sqlcipher3-binary)
- ? Platforms: Windows, Linux (x86_64, ARM supported)

Disadvantages:
- ? **Key must be provided at every connection** (no persistent "unlock" of the file)
- ? If key is in keychain + malware can access keychain, DB is decrypted
- ? Performance: ~10-30% slower than plaintext SQLite (cipher overhead)
- ?? **License**: BSD 3-Clause (Zetetic, Community Edition)?permissive, safe

Query performance on encrypted data:
- ? WHERE clauses: No degradation (encrypted index comparison works)
- ? Aggregations (SUM, COUNT, AVG, GROUP BY): Work transparently on numeric fields
- ? ORDER BY, LIMIT: Full support
- ? Full-text search: May need plugin integration

Reference: [How SQLCipher Compares to Other SQLite Encryption](https://www.zetetic.net/sqlcipher/comparison/)

Option B: Python cryptography (Fernet) + Application-Level Encryption

What it does:
- Encrypt sensitive columns (amounts, merchant names) before INSERT
- Decrypt after SELECT
- Queries on encrypted columns (WHERE, aggregates) **DO NOT WORK**?must fetch and decrypt in app

Disadvantages:
- ? **Breaks SQL queryability** (cannot `SELECT WHERE amount > 1000` on encrypted amounts)
- ? Requires app-level filtering (fetch all rows, decrypt, filter in Python ? slow)
- ? License: Apache-2.0 OR BSD-3-Clause

Verdict: **Not suitable for this use case** (breaks SQL queries, defeats purpose).

Option C: No Encryption + FDE Only

Profile:
- ? Simplicity: Standard sqlite3 API
- ? Performance: Zero overhead
- ? Fails cloud backup scenario (unencrypted in cloud)
- ? Fails malware read scenario (malware can read plaintext DB from filesystem)

Verdict: **Insufficient** if backups are cloud-synced.

PART E: NIST & OWASP RECOMMENDATIONS

NIST SP 800-111: Guide to Storage Encryption Technologies

```
**Key guidance**:
- FDE (BitLocker, LUKS) protects against physical loss but not software-level threats
- Database-level encryption complements FDE for additional defense-in-depth
- **Threat layers**: FDE addresses hardware theft; application-level encryption addresses
malware/backups

**Reference**: [NIST SP 800-111](https://csrc.nist.gov/pubs/sp/800/111/final)
```

OWASP Password Storage / Key Derivation

```
**Applicable if using passphrase-derived keys**:
- **Recommended**: Argon2id (19 MiB memory, 2 iterations, 1 parallelism)
- **Fallback**: PBKDF2-HMAC-SHA256 (600k iterations, FIPS-140 compatible)
- **Avoid**: MD5, SHA-1 (too fast; GPU-breakable)

**Reference**: [OWASP Password Storage Cheat
Sheet](https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html)

---
```

PART F: RECOMMENDATIONS FOR MUNEEM

Recommended Architecture

```
**Layer 1: OS Full-Disk Encryption (Mandatory)**
- Windows: BitLocker (built-in; enable via Group Policy or Settings)
- Linux: LUKS (built-in; enable at install or via cryptsetup)
- **Protects**: Stolen/discarded hardware
- **Cost**: Minimal (integrated; no performance hit if hardware-accelerated)

**Layer 2: SQLCipher Database Encryption (Recommended)**
- Encrypt muneem.db with SQLCipher
- Store key in OS keyring (Windows Credential Manager / Linux Secret Service)
- **Protects**: Cloud backups, malware reads (adds friction)
- **Implementation**:
  ```python
  import keyring
  import pysqlcipher3 as sqlite3

  # Setup (first time)
  key_hex = keyring.get_password("muneem", "db_key")
  if not key_hex:
      import secrets
      key_hex = secrets.token_hex(32)  # 256-bit random
      keyring.set_password("muneem", "db_key", key_hex)

  # Connection
  conn = sqlite3.connect("muneem.db")
  conn.execute(f"PRAGMA key = \"x'{key_hex}';\"")
  ```
- **Cost**: ~15-20% performance overhead (acceptable for personal finance app)

Layer 3 (Optional): Passphrase-Backed Key Derivation
- If user wants additional assurance: derive key from passphrase using Argon2id
- Trade-off: user types passphrase on app startup (friction)
- **Not required** for baseline threat model
```

Key Management Best Practices

```
1. **Never hardcode keys** in source, config files, or version control
2. **Never store passphrases** in plaintext config
3. **Use OS keyring** (keyring library) for random keys
4. **Rotate keys** (optional): If compromise suspected, re-encrypt database with new key
5. **Backup encrypted**: Always encrypt backups separately (e.g., encrypted 7-zip or tarball
```

before cloud sync)

Dependency Review: License Compliance

Dependency	License	Status
pysqlcipher3 or **sqlcipher3-binary**	BSD-3-Clause	? Permissive, safe
cryptography	Apache-2.0 OR BSD-3-Clause	? Permissive, safe
keyring (v25.0+)	MIT	? Permissive, safe
OpenSSL (underlying SQLCipher)	Apache-2.0 / dual OpenSSL	? Acceptable per constraints

All dependencies meet the requirement: MIT / BSD / Apache-2.0 / ISC / HPND. No GPL/LGPL/AGPL detected.

PART G: THREAT MODEL SUMMARY TABLE

Threat	FDE Only	FDE + SQLCipher	Risk Level	Why?
Stolen/discarded disk	? Mitigated	? Mitigated	Low	FDE sufficient; SQLCipher adds no marginal value if key is on same disk
Cloud backup breach	? Exposed	? Mitigated	**HIGH**	SQLCipher essential; FDE doesn't protect cloud storage
Malware (user-level)	? Exposed	?? Slows attack	**HIGH**	Neither fully stops this; SQLCipher raises bar (extract key from keychain required)
Shoulder surfing	? Good	? Better	Low	Keyring approach hides key retrieval
Multi-user machine	? Protects from other users	? Adds defense	Medium	FDE + SQLCipher provides strong isolation
Lost laptop (powered on)	? FDE protects if locked	?? SQLCipher helps if DB not open	Medium	If laptop is suspended and FDE is locked, both work. If running, FDE volume is decrypted.

PART H: DECISION: DB-LEVEL ENCRYPTION ADDS REAL SECURITY VALUE?

Yes, but conditionally:

1. **Cloud backup scenario**: CRITICAL. Without SQLCipher, cloud storage contains plaintext sensitive data. ? **Real security value.**

2. **Malware scenario**: MODEST. Malware can still extract key from keychain, but:
   - Requires malware to implement keyring extraction (not trivial)
   - Adds defense-in-depth: attacker must break into two systems (OS keychain + SQLCipher)
   - Delays breach (attacker must read HMAC-protected pages, not raw bytes)
   - ? **Real but limited security value.**

3. **Stolen disk scenario**: MINIMAL. If key is on the same disk (in keychain), attacker can extract it offline.
   - Exception: If user enforces strong BitLocker passphrase AND attacker has no offline dictionary tools, SQLCipher adds no marginal value
   - ? **No real security value** (FDE is sufficient).

4. **Defense-in-depth principle**: STRONG ENDORSEMENT.
   - Layered encryption is industry best practice (NIST SP 800-111, Oracle, Azure guidance)
   - Cost is acceptable (20% performance, minimal code complexity)
   - ? **Recommended.**

FINAL RECOMMENDATION FOR MUNEEM

Implementation Path (Priority Order)

```
**Phase 1 (Baseline Security - IMMEDIATE)**
1. Mandate OS FDE on user's machine (user responsibility; document in README)
2. Integrate SQLCipher + keyring-based key storage
3. Test on Windows + Linux (verify wheels work)
4. Update docs: explain threat model & encryption architecture

**Phase 2 (Enhanced Security - OPTIONAL)**
1. Add optional passphrase-based key derivation (Argon2id) for advanced users
2. Document backup encryption best practices (encrypt tarball before uploading)
3. Add key rotation capability if user suspects compromise

**Phase 3 (Future)**
1. Monitor for newer encryption standards (AES-GCM if SQLCipher adopts)
2. Consider hardware-backed key storage (TPM integration) for very high-threat scenarios
```

Code Snippet: Recommended Setup

```python
# src/muneem/db.py (updated)
import sqlite3
from pathlib import Path
import keyring
import secrets

def get_or_create_db_key() -> str:
    """Get database encryption key from OS keychain."""
    key_hex = keyring.get_password("muneem", "db_encryption_key")
    if key_hex is None:
        # First use: generate and store random 256-bit key
        key_hex = secrets.token_hex(32)
        keyring.set_password("muneem", "db_encryption_key", key_hex)
    return key_hex

def connect(db_path: Path | str):
    """Open encrypted SQLCipher connection."""
    path = str(db_path)
    if path != ":memory:":
        Path(path).parent.mkdir(parents=True, exist_ok=True)

    # Import pysqlcipher3 instead of stdlib sqlite3
    import pysqlcipher3 as sqlite3

    conn = sqlite3.connect(path)

    # Unlock database with key from keyring
    key_hex = get_or_create_db_key()
    conn.execute(f"PRAGMA key = \"x'{key_hex}';\"")

    conn.row_factory = sqlite3.Row
    conn.execute("PRAGMA foreign_keys = ON")
    return conn
```

SOURCES CITED

- [Zetetic: SQLCipher Comparison](https://www.zetetic.net/sqlcipher/comparison/)
- [OWASP Password Storage Cheat Sheet](https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html)
- [Charles Leifer: Encrypted SQLite Databases with Python and SQLCipher](https://charlesleifer.com/blog/encrypted-sqlite-databases-with-python-and-sqlcipher/)
- [NIST SP 800-111: Guide to Storage Encryption

Technologies](https://csrc.nist.gov/pubs/sp/800/111/final)
- [PyPI: sqlcipher3-wheels](https://pypi.org/project/sqlcipher3-wheels/)
- [PyPI: cryptography](https://pypi.org/project/cryptography/)
- [PyPI: keyring](https://pypi.org/project/keyring/)
- [SQLCipher License Information](https://www.zetetic.net/sqlcipher/license/)
- [Guardian Project: SQLCipher](https://guardianproject.info/code/sqlcipher/)
- [Medium: Python Keyring Backends](https://johal.in/python-keyring-backends-secretservice-windows-credential-manager-support-2025/)
- [Medium: BitLocker vs LUKS vs FileVault](https://medium.com/@cybersecuritystephen/disk-encryption-tools-and-which-is-best-for-you-d300a8bbfe75/)
- [SecurityBoulevard: Defense-in-Depth with Persistent and Transparent Data Encryption](https://securityboulevard.com/2026/03/defense-in-depth-combining-persistent-and-transparent-data-encryption/)